Anomaly Detection in Financial Transactions

Ready-to-run Colab pipeline for:

- data loading and basic cleaning
- preprocessing
- training 6 anomaly detection models
- evaluation with confusion matrix, precision, recall, F1-score, ROC AUC
- result tables and plots for a diploma project

```
import warnings
warnings.filterwarnings("ignore")

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import (
    precision_score, recall_score, f1_score,
    roc_auc_score, confusion_matrix, ConfusionMatrixDisplay
)

from sklearn.ensemble import IsolationForest
from sklearn.svm import OneClassSVM
from sklearn.neighbors import LocalOutlierFactor
from sklearn.covariance import EllipticEnvelope
from sklearn.cluster import KMeans
```

```
df = pd.read_csv('/content/creditcard.csv')
df.head()
```

	Time	V1	V2	V3	V4	V5	V6	V7	V8	V9	...	V21	V22
0	0	-1.359807	-0.072781	2.536347	1.378155	-0.338321	0.462388	0.239599	0.098698	0.363787	...	-0.018307	0.277838
1	0	1.191857	0.266151	0.166480	0.448154	0.060018	-0.082361	-0.078803	0.085102	-0.255425	...	-0.225775	-0.638672
2	1	-1.358354	-1.340163	1.773209	0.379780	-0.503198	1.800499	0.791461	0.247676	-1.514654	...	0.247998	0.771679
3	1	-0.966272	-0.185226	1.792993	-0.863291	-0.010309	1.247203	0.237609	0.377436	-1.387024	...	-0.108300	0.005274
4	2	-1.158233	0.877737	1.548718	0.403034	-0.407193	0.095921	0.592941	-0.270533	0.817739	...	-0.009431	0.798278

5 rows × 31 columns

```
print("Shape:", df.shape)
print("\nMissing values:")
print(df.isnull().sum())

print("\nDuplicate rows:", df.duplicated().sum())

print("\nClass distribution:")
print(df['Class'].value_counts())

print("\nClass distribution (%):")
print(df['Class'].value_counts(normalize=True) * 100)
```

```
Shape: (128821, 31)

Missing values:
Time      0
V1        0
V2        0
V3        0
V4        0
V5        0
V6        0
V7        0
V8        0
V9        0
V10       0
V11       0
V12       0
V13       0
V14       0
```

```
V15       0
V16       1
V17       1
V18       1
V19       1
V20       1
V21       1
V22       1
V23       1
V24       1
V25       1
V26       1
V27       1
V28       1
Amount    1
Class     1
dtype: int64

Duplicate rows: 489

Class distribution:
Class
0.0    128559
1.0       261
Name: count, dtype: int64

Class distribution (%):
Class
0.0    99.797392
1.0     0.202608
Name: proportion, dtype: float64
```

```
df = df.drop_duplicates().reset_index(drop=True)
print('Shape after dropping duplicates:', df.shape)
df = df.dropna(subset=['Class']).reset_index(drop=True)
```

```
Shape after dropping duplicates: (128332, 31)
```

```
X = df.drop('Class', axis=1)
y = df['Class']
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

print("Train shape:", X_train.shape)
print("Test shape:", X_test.shape)

print("\nTrain class distribution:")
print(y_train.value_counts())

print("\nTest class distribution:")
print(y_test.value_counts())
```

```
Train shape: (102664, 30)
Test shape: (25667, 30)

Train class distribution:
Class
0.0    102459
1.0       205
Name: count, dtype: int64

Test class distribution:
Class
0.0    25616
1.0       51
Name: count, dtype: int64
```

```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

```
X_train_normal = X_train_scaled[y_train == 0]
print("Normal train transactions:", X_train_normal.shape[0])
print("Fraud train transactions:", (y_train == 1).sum())
```

```
Normal train transactions: 102459
Fraud train transactions: 205
```

```
contamination = y_train.mean()
print('Contamination:', contamination)
```

```
Contamination: 0.001996805111821086
```

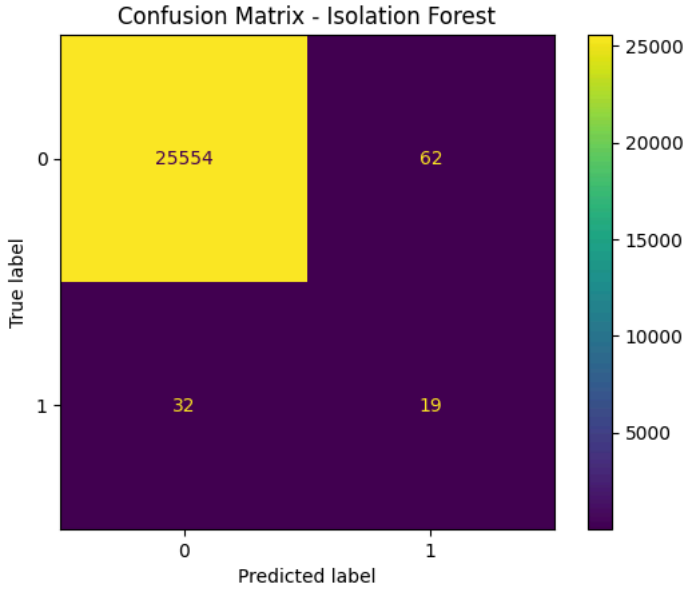
```
rng = np.random.RandomState(42)

def sample_rows(X, n, random_state=42):
    if len(X) <= n:
        return X
    idx = rng.choice(len(X), size=n, replace=False)
    return X[idx]

X_train_normal_small_svm = sample_rows(X_train_normal, 20000)
X_train_normal_small_lof = sample_rows(X_train_normal, 20000)
X_train_normal_small_ee = sample_rows(X_train_normal, 10000)

print("SVM sample:", X_train_normal_small_svm.shape)
print("LOF sample:", X_train_normal_small_lof.shape)
print("Elliptic sample:", X_train_normal_small_ee.shape)
```

```
SVM sample: (20000, 30)
LOF sample: (20000, 30)
Elliptic sample: (10000, 30)
```

```
results = []

def evaluate_predictions(model_name, y_true, y_pred, y_score):
    precision = precision_score(y_true, y_pred, zero_division=0)
    recall = recall_score(y_true, y_pred, zero_division=0)
    f1 = f1_score(y_true, y_pred, zero_division=0)

    try:
        roc_auc = roc_auc_score(y_true, y_score)
    except Exception:
        roc_auc = np.nan

    cm = confusion_matrix(y_true, y_pred)

    results.append({
        'Model': model_name,
        'Precision': precision,
        'Recall': recall,
        'F1-score': f1,
        'ROC AUC': roc_auc,
        'TN': cm[0, 0],
        'FP': cm[0, 1],
        'FN': cm[1, 0],
        'TP': cm[1, 1]
    })

    print(f"\n{model_name}")
    print("Precision:", round(precision, 4))
    print("Recall:", round(recall, 4))
    print("F1-score:", round(f1, 4))
    print("ROC AUC:", round(roc_auc, 4) if not np.isnan(roc_auc) else "N/A")

    disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=[0, 1])
    disp.plot()
    plt.title(f'Confusion Matrix - {model_name}')
    plt.show()
```

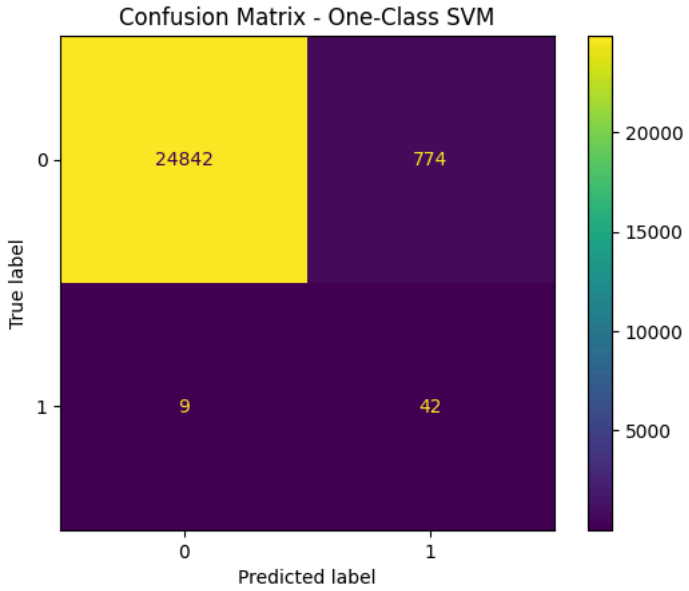
```
model_if = IsolationForest(
    contamination=contamination,
    random_state=42,
    n_estimators=100
)

model_if.fit(X_train_normal)

pred_if = model_if.predict(X_test_scaled)
y_pred_if = np.where(pred_if == -1, 1, 0)

score_if = -model_if.decision_function(X_test_scaled)

evaluate_predictions("Isolation Forest", y_test, y_pred_if, score_if)
```

```
Isolation Forest
Precision: 0.2346
Recall: 0.3725
F1-score: 0.2879
ROC AUC: 0.93
```




```
model_svm = OneClassSVM(
    kernel='rbf',
    nu=contamination,
    gamma='scale'
)

model_svm.fit(X_train_normal_small_svm)

pred_svm = model_svm.predict(X_test_scaled)
y_pred_svm = np.where(pred_svm == -1, 1, 0)

score_svm = -model_svm.decision_function(X_test_scaled)

evaluate_predictions("One-Class SVM", y_test, y_pred_svm, score_svm)
```

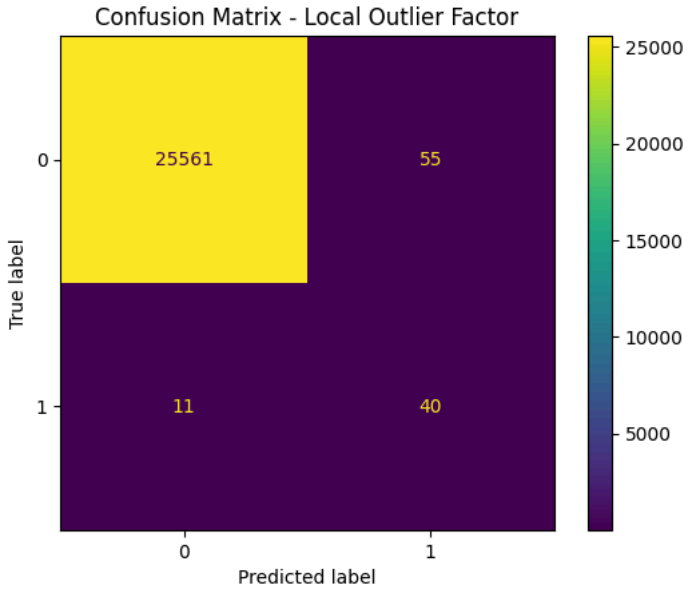
```
One-Class SVM
Precision: 0.0515
Recall: 0.8235
F1-score: 0.0969
ROC AUC: 0.9353
```




```
model_lof = LocalOutlierFactor(
    n_neighbors=20,
    contamination=contamination,
    novelty=True
)
```

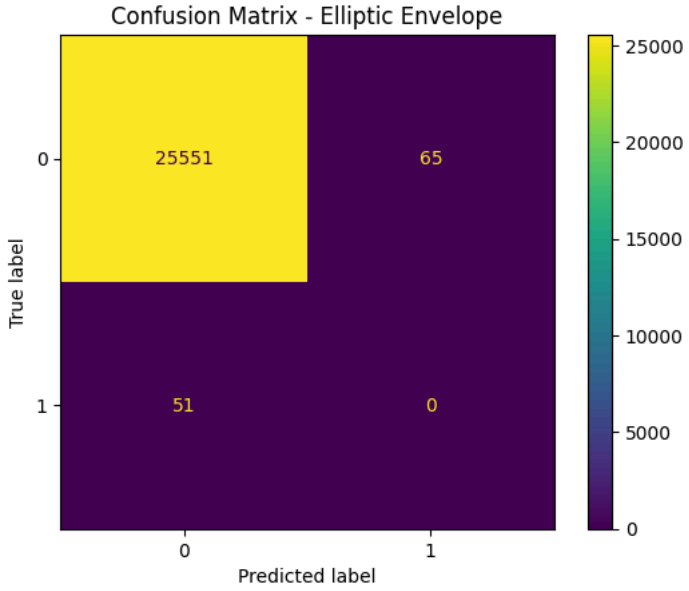
```
model_lof.fit(X_train_normal_small_lof)

pred_lof = model_lof.predict(X_test_scaled)
y_pred_lof = np.where(pred_lof == -1, 1, 0)

score_lof = -model_lof.decision_function(X_test_scaled)

evaluate_predictions("Local Outlier Factor", y_test, y_pred_lof, score_lof)
```

```
Local Outlier Factor
Precision: 0.4211
Recall: 0.7843
F1-score: 0.5479
ROC AUC: 0.9569
```



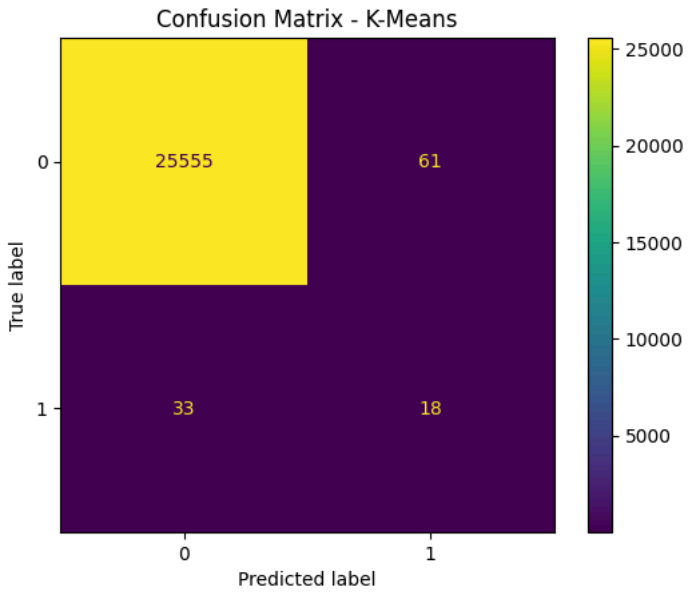

```
model_ee = EllipticEnvelope(
    contamination=contamination,
    random_state=42
)

model_ee.fit(X_train_normal_small_ee)

pred_ee = model_ee.predict(X_test_scaled)
y_pred_ee = np.where(pred_ee == -1, 1, 0)

score_ee = -model_ee.decision_function(X_test_scaled)

evaluate_predictions("Elliptic Envelope", y_test, y_pred_ee, score_ee)
```

```
Elliptic Envelope
Precision: 0.0
Recall: 0.0
F1-score: 0.0
ROC AUC: 0.804
```




```
model_kmeans = KMeans(
    n_clusters=2,
    random_state=42,
    n_init=10
)

model_kmeans.fit(X_train_normal)

train_distances = np.min(model_kmeans.transform(X_train_normal), axis=1)
test_distances = np.min(model_kmeans.transform(X_test_scaled), axis=1)

threshold_kmeans = np.percentile(train_distances, 100 * (1 - contamination))

y_pred_kmeans = (test_distances > threshold_kmeans).astype(int)
score_kmeans = test_distances

evaluate_predictions("K-Means", y_test, y_pred_kmeans, score_kmeans)
```

```
K-Means
Precision: 0.2278
Recall: 0.3529
F1-score: 0.2769
ROC AUC: 0.9366
```




```
import tensorflow as tf
from tensorflow.keras.models import Model
```

```
from tensorflow.keras.layers import Input, Dense
from tensorflow.keras.callbacks import EarlyStopping

tf.random.set_seed(42)
```

```
input_dim = X_train_normal.shape[1]

input_layer = Input(shape=(input_dim,))
encoded = Dense(16, activation='relu')(input_layer)
encoded = Dense(8, activation='relu')(encoded)
decoded = Dense(16, activation='relu')(encoded)
decoded = Dense(input_dim, activation='linear')(decoded)

autoencoder = Model(inputs=input_layer, outputs=decoded)
autoencoder.compile(optimizer='adam', loss='mse')

autoencoder.summary()
```

Model: "functional"

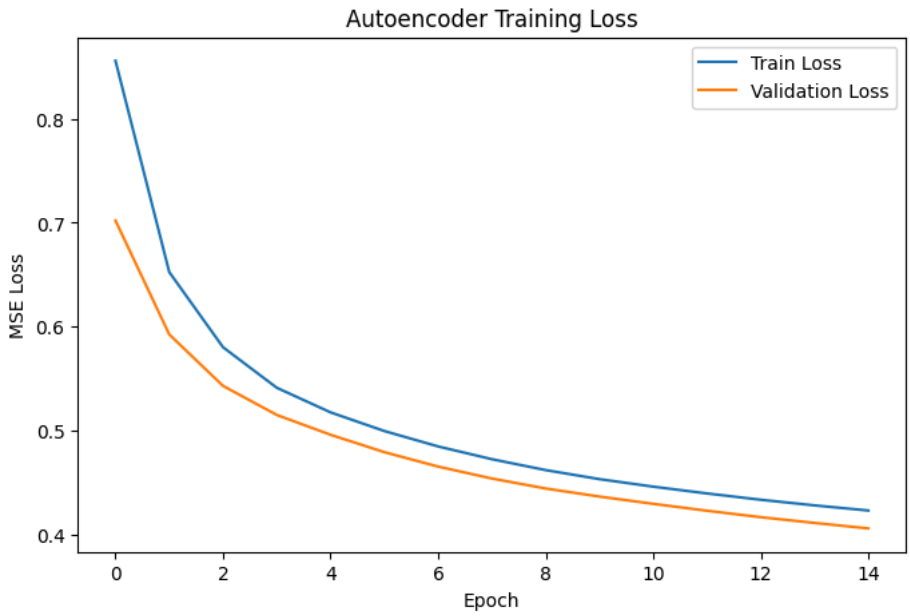
Layer (type)	Output Shape	Param #
input_layer (InputLayer)	(None, 30)	0
dense (Dense)	(None, 16)	496
dense_1 (Dense)	(None, 8)	136
dense_2 (Dense)	(None, 16)	144
dense_3 (Dense)	(None, 30)	510

Total params: 1,286 (5.02 KB)
Trainable params: 1,286 (5.02 KB)
Non-trainable params: 0 (0.00 B)

```
early_stop = EarlyStopping(
    monitor='val_loss',
    patience=3,
    restore_best_weights=True
)

history = autoencoder.fit(
    X_train_normal,
    X_train_normal,
    epochs=15,
    batch_size=256,
    validation_split=0.2,
    shuffle=True,
    callbacks=[early_stop],
    verbose=1
)
```

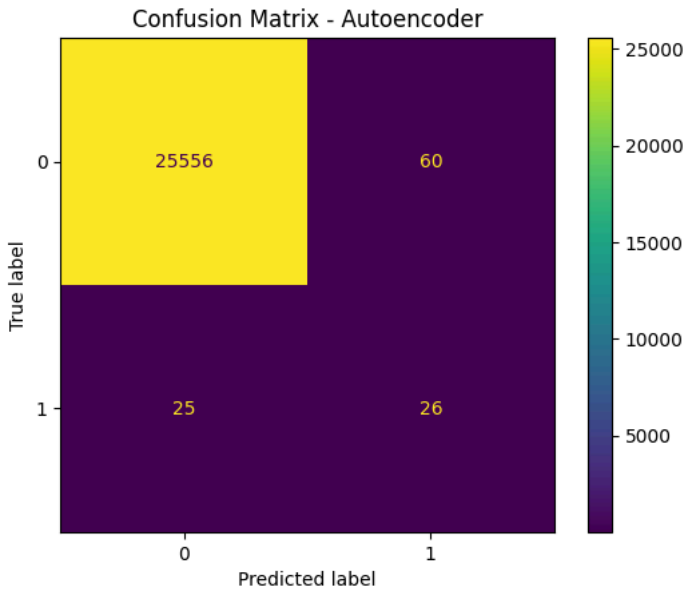
```
Epoch 1/15
321/321 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.8560 - val_loss: 0.7021
Epoch 2/15
321/321 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.6525 - val_loss: 0.5924
Epoch 3/15
321/321 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.5801 - val_loss: 0.5429
Epoch 4/15
321/321 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.5411 - val_loss: 0.5149
Epoch 5/15
321/321 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.5174 - val_loss: 0.4958
Epoch 6/15
321/321 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.4994 - val_loss: 0.4791
Epoch 7/15
321/321 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.4846 - val_loss: 0.4652
Epoch 8/15
321/321 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.4723 - val_loss: 0.4537
Epoch 9/15
321/321 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - loss: 0.4618 - val_loss: 0.4441
Epoch 10/15
321/321 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - loss: 0.4531 - val_loss: 0.4363
Epoch 11/15
321/321 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - loss: 0.4459 - val_loss: 0.4293
Epoch 12/15
321/321 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.4393 - val_loss: 0.4227
Epoch 13/15
321/321 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.4333 - val_loss: 0.4165
Epoch 14/15
321/321 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.4278 - val_loss: 0.4110
Epoch 15/15
321/321 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - loss: 0.4229 - val_loss: 0.4057
```

```
plt.figure(figsize=(8, 5))
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title('Autoencoder Training Loss')
plt.xlabel('Epoch')
plt.ylabel('MSE Loss')
plt.legend()
plt.show()
```




```
reconstructions_train = autoencoder.predict(X_train_normal, verbose=0)
mse_train = np.mean(np.power(X_train_normal - reconstructions_train, 2), axis=1)

reconstructions_test = autoencoder.predict(X_test_scaled, verbose=0)
mse_test = np.mean(np.power(X_test_scaled - reconstructions_test, 2), axis=1)

threshold_ae = np.percentile(mse_train, 100 * (1 - contamination))

y_pred_ae = (mse_test > threshold_ae).astype(int)
score_ae = mse_test

evaluate_predictions("Autoencoder", y_test, y_pred_ae, score_ae)
```

```
Autoencoder
Precision: 0.3023
Recall: 0.5098
F1-score: 0.3796
ROC AUC: 0.9429
```



```
results_df = pd.DataFrame(results)
results_df = results_df.sort_values(by='F1-score', ascending=False).reset_index(drop=True)
results_df
```

	Model	Precision	Recall	F1-score	ROC AUC	TN	FP	FN	TP
0	Local Outlier Factor	0.421053	0.784314	0.547945	0.956924	25561	55	11	40
1	Autoencoder	0.302326	0.509804	0.379562	0.942915	25556	60	25	26
2	Isolation Forest	0.234568	0.372549	0.287879	0.930044	25554	62	32	19
3	K-Means	0.227848	0.352941	0.276923	0.936638	25555	61	33	18
4	One-Class SVM	0.051471	0.823529	0.096886	0.935342	24842	774	9	42
5	Elliptic Envelope	0.000000	0.000000	0.000000	0.804030	25551	65	51	0

Next steps: Generate code with `results_df` New interactive sheet

```
results_table = results_df.copy()
for col in ['Precision', 'Recall', 'F1-score', 'ROC AUC']:
    results_table[col] = results_table[col].round(4)
results_table
```

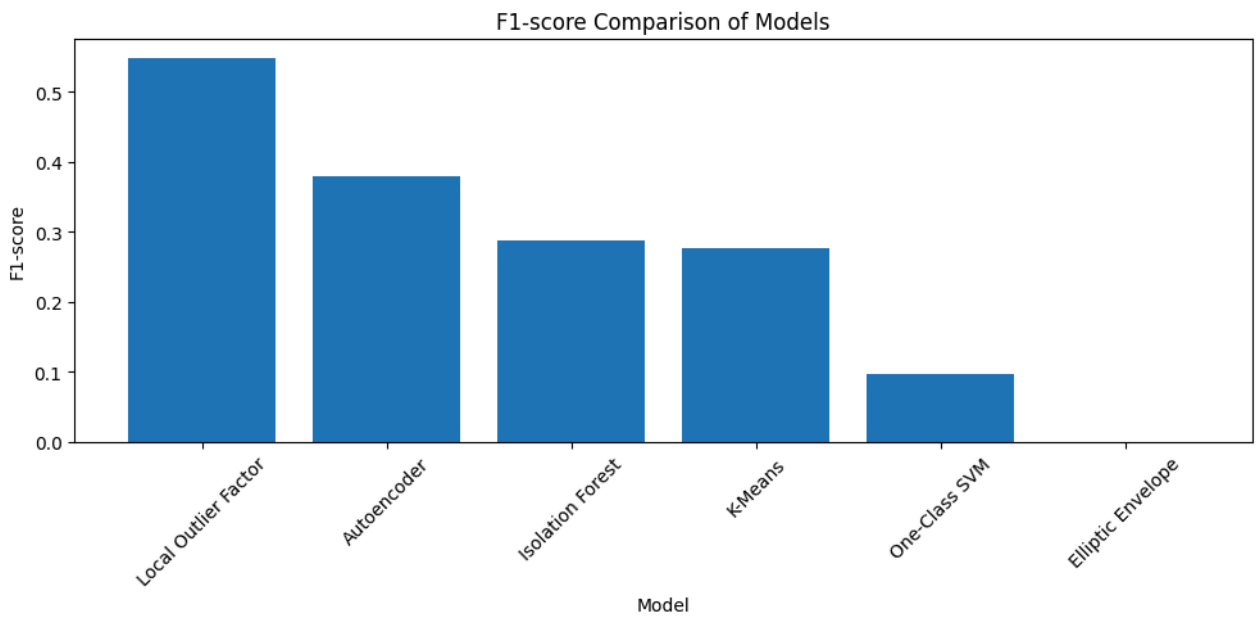
	Model	Precision	Recall	F1-score	ROC AUC	TN	FP	FN	TP
0	Local Outlier Factor	0.4211	0.7843	0.5479	0.9569	25561	55	11	40
1	Autoencoder	0.3023	0.5098	0.3796	0.9429	25556	60	25	26
2	Isolation Forest	0.2346	0.3725	0.2879	0.9300	25554	62	32	19
3	K-Means	0.2278	0.3529	0.2769	0.9366	25555	61	33	18
4	One-Class SVM	0.0515	0.8235	0.0969	0.9353	24842	774	9	42
5	Elliptic Envelope	0.0000	0.0000	0.0000	0.8040	25551	65	51	0

Next steps: Generate code with `results_table` New interactive sheet

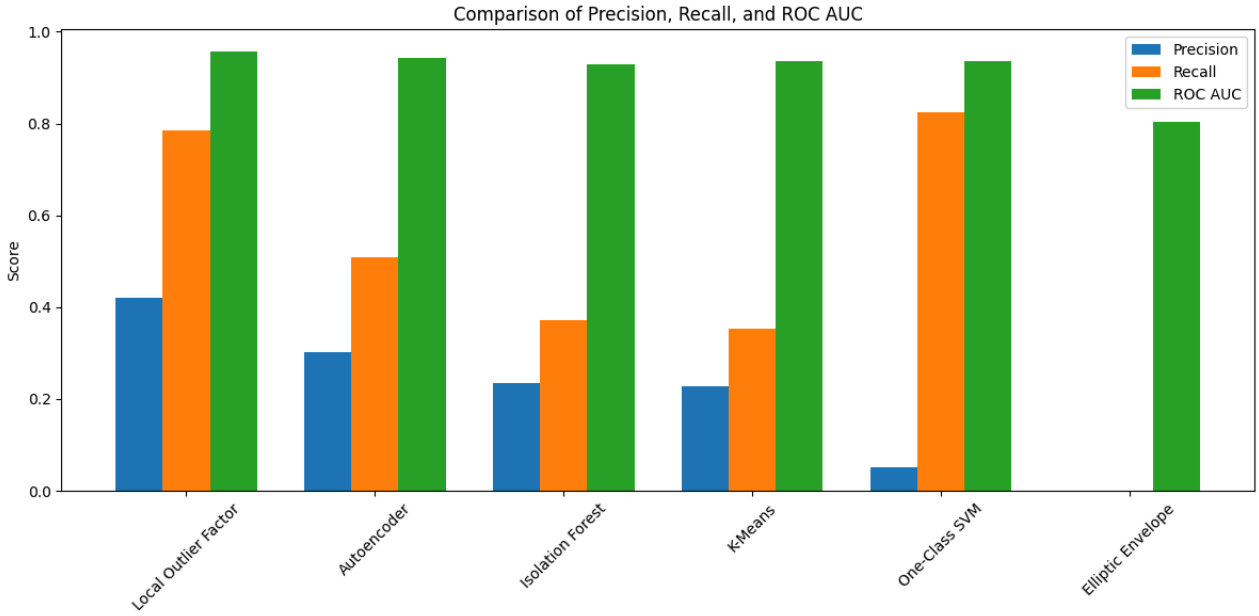
```
results_table.to_csv('/content/model_comparison_results.csv', index=False)
print('Saved: /content/model_comparison_results.csv')
```

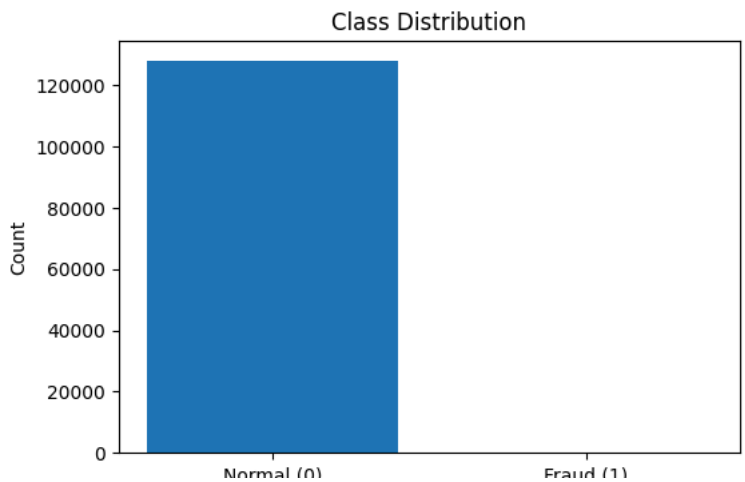
```
Saved: /content/model_comparison_results.csv
```

```
plt.figure(figsize=(10, 5))
plt.bar(results_df['Model'], results_df['F1-score'])
plt.title('F1-score Comparison of Models')
plt.xlabel('Model')
plt.ylabel('F1-score')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



```
x = np.arange(len(results_df))
width = 0.25

plt.figure(figsize=(12, 6))
plt.bar(x - width, results_df['Precision'], width, label='Precision')
plt.bar(x, results_df['Recall'], width, label='Recall')
plt.bar(x + width, results_df['ROC AUC'], width, label='ROC AUC')

plt.xticks(x, results_df['Model'], rotation=45)
plt.ylabel('Score')
plt.title('Comparison of Precision, Recall, and ROC AUC')
plt.legend()
plt.tight_layout()
plt.show()
```




```
class_counts = df['Class'].value_counts().sort_index()

plt.figure(figsize=(6, 4))
plt.bar(['Normal (0)', 'Fraud (1)'], class_counts.values)
plt.title('Class Distribution')
plt.ylabel('Count')
plt.show()
```